IT STUDY HUB

COMPLETE R PROGRAMMING SERIES

MODULE 6: DATA VISUALIZATION WITH R

Authors:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

WELCOME TO MODULE 6

About This Module

Welcome to Module 6 of the **IT Study Hub Complete R Programming Series**. Up until this point, we have operated entirely in the realm of raw text and numbers. You have learned how to import, clean, filter, and mathematically manipulate datasets using robust coding pipelines. However, raw data fails to tell a compelling story to human stakeholders.

This module transitions you from data wrangling to Data Storytelling. You will learn the mathematics of geometry, translating abstract numbers into pixels, lines, and colors. We begin with Base R's rapid, functional plotting system to instantly inspect your data. Then, we dedicate the second half of the module to `ggplot2`—the undisputed industry standard for data visualization. You will master the 'Grammar of Graphics', learning to build publication-ready, multi-layered, and aesthetically flawless charts piece by piece.

IT STUDY HUB

Learning Outcomes

By the completion of this comprehensive module, you will be able to:

- Identify the correct chart type (Scatter, Bar, Histogram, Box, Line) based on underlying variable types (Categorical vs Continuous).
- Generate rapid exploratory plots using Base R's built-in `plot()` framework.
- Customize axes, titles, bounds, colors, and legends for high readability.
- Comprehend the layered architecture of the **Grammar of Graphics**.
- Construct complex `ggplot2` visual pipelines using Data, Aesthetics, and Geometries.
- Distinguish between mapping dynamic data to an aesthetic versus setting fixed visual parameters.
- Utilize Themes and Facets to generate beautiful, sub-divided, presentation-ready graphics.

Module Coverage

This handbook thoroughly covers the following core topics:

1. Introduction to data visualization
2. Base R plotting
3. Scatter plots
4. Bar charts
5. Histograms
6. Box plots
7. Line graphs
8. Pie charts
9. Customizing plots
10. Titles, labels and legends
11. Introduction to ggplot2
12. Grammar of Graphics
13. ggplot() and Aesthetics
14. Geometries
15. Themes and Facets

IT STUDY HUB

MAIN NOTES

1. Introduction to Data Visualization

CONCEPT:
Data visualization is the graphical representation of information and data to highlight patterns, trends, and outliers.

Deep Dive Explanation:
Human brains process visuals up to 60,000 times faster than text. Raw data in spreadsheets is practically invisible to the human eye when searching for trends. Data visualization transforms abstract numbers into shapes, colors, and spatial arrangements. In R, visualization happens through Base R (built-in plotting) or modern packages like `ggplot2`.

Real-Life Example:
Looking at 10,000 rows of daily temperature data makes it impossible to see seasonality. Plotting it as a line graph instantly reveals the summer spikes and winter drops.

Syntax Structure:

```
# Core R plotting frameworks
plot(x, y)          # Base R
ggplot(data, aes()) # ggplot2
```

R Code Example

```
# Demonstration of Anscombe's Quartet concept
# (Identical summary statistics, completely different visual plots)
x <- c(10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5)
y <- c(8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68)

print(paste("Mean X:", round(mean(x),2), "| Mean Y:", round(mean(y), 2)))
```

Console / Visual Output

```
[1] "Mean X: 9 | Mean Y: 7.5"
```

Line-by-Line Explanation:

These basic statistics (mean, variance) hide the true distribution. Only by plotting `x` and `y` can we see the actual relationship (a linear scatter).

When to Use:

Always visualize your data *before* running statistical models to check for outliers or non-linear relationships.

Common Mistakes:

Relying purely on summary statistics (`mean()`, `sd()`) without visualizing the data first, leading to false assumptions.

Best Practices:

Start with exploratory data analysis (EDA) using simple, fast Base R plots before spending time building highly polished `ggplot2` charts for presentations.

Practice Challenge:
Print the `summary()` of the built-in `cars` dataset to see numeric summaries, then imagine how a plot would better display the relationship between speed and distance.

Variable Mapping Basics: X and Y Axes



2. Base R Plotting (Scatter Plots)

CONCEPT:
The plot() function is Base R's workhorse, primarily used to create scatter plots showing the relationship between two continuous variables.

Deep Dive Explanation:
Base R comes with visualization capabilities out-of-the-box. The `plot(x, y)` function takes two vectors of equal length. Each pair of values becomes an (X, Y) coordinate represented by a dot. Scatter plots are essential for identifying correlations (positive, negative, or none) and isolating outliers.

Real-Life Example:
Plotting an employee's Years of Experience (X-axis) against their Salary (Y-axis) to see if more experience correlates with higher pay.

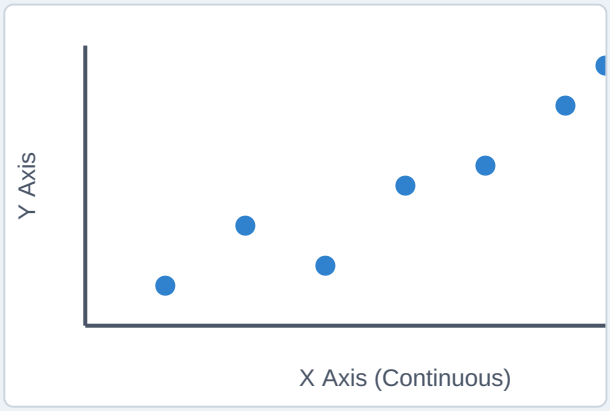
Syntax Structure:

```
plot(x = vector1, y = vector2)
```

R Code Example

```
# Using the built-in 'mtcars' dataset
# x = Weight of car (wt), y = Miles per gallon (mpg)
plot(x = mtcars$wt,
     y = mtcars$mpg,
     main = "Car Weight vs. MPG",
     xlab = "Weight (1000 lbs)",
     ylab = "Miles per Gallon")
```

Visualization Output



(A window opens displaying a scatter plot showing a negative correlation: as weight increases, mpg decreases.)

Line-by-Line Explanation:
We passed the `wt` column as the X coordinate and `mpg` as the Y coordinate. The `main`,

`xlab`, and `ylab` arguments add human-readable text to the chart. Every dot represents one specific car.

When to Use:
Use scatter plots strictly when comparing TWO CONTINUOUS (numeric) variables.

Common Mistakes:
Trying to use `plot()` with a categorical variable on the X-axis and a continuous on the Y-axis. R will usually try to draw a boxplot or a messy scatter instead.

Best Practices:
If you have thousands of overlapping dots (overplotting), use transparency (`col = rgb(0,0,0,0.2)`) so dense areas appear darker.

Practice Challenge:
Use `plot()` to create a scatter plot of `disp` (Engine displacement) vs `hp` (Horsepower) using the `mtcars` dataset.

3. Bar Charts

CONCEPT:
Bar charts use rectangular bars to represent the frequency, count, or magnitude of categorical data.

Deep Dive Explanation:
A bar chart compares distinct categories. In Base R, you use `barplot()`. However, `barplot()` does NOT automatically count your data. If you have a raw list of categories (e.g., "Red", "Blue", "Red"), you must first compress it into a frequency table using the `table()` function before passing it into `barplot()`.

Real-Life Example:
Comparing the total total sales revenue generated across four different global regions: North America, Europe, Asia, and Africa.

Syntax Structure:

```
counts <- table(categorical_vector)
barplot(counts)
```

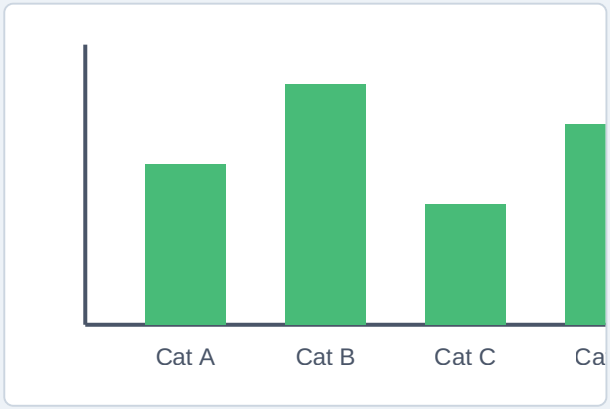
R Code Example

```
# Simulating a dataset of student grades
grades <- c("A", "B", "A", "C", "B", "B", "A", "F")

# 1. Count the frequencies
grade_counts <- table(grades)
print(grade_counts)

# 2. Plot the bar chart
barplot(grade_counts,
        col = "steelblue",
        main = "Grade Distribution")
```

Visualization Output




```
grades A B C F 3 3 1 1 (A bar chart renders with 4 bars of varying heights
                        corresponding to the counts.)
```

Line-by-Line Explanation:

`table()` counts the occurrences of each unique string. `barplot()` takes those counts and draws heights. The `col` argument changes the fill color.

When to Use:

Use bar charts when comparing quantities across distinct CATEGORICAL groupings.

Common Mistakes:

Passing a raw, uncounted vector into `barplot()`. It will either crash or draw a bar for every single row in your dataset.

Best Practices:

Order your bars logically. If the categories don't have a natural order (like Grades do), sort the bars from highest frequency to lowest frequency to make comparisons easier.

Practice Challenge:

Create a vector of 10 coin flips (e.g., "Heads", "Tails"). Use `table()` and `barplot()` to visualize the results.

4. Histograms

CONCEPT:
Histograms visualize the distribution of a single continuous variable by grouping numbers into contiguous 'bins'.

Deep Dive Explanation:
While a bar chart shows distinct categories, a histogram shows a continuous numeric scale. R's `hist()` function takes a single numeric vector, divides the range of numbers into equal-sized buckets (bins), and counts how many data points fall into each bucket. The bars in a histogram touch each other to indicate that the X-axis is a continuous number line.

Real-Life Example:
Visualizing the distribution of ages in a city. The bins might be 0-10, 10-20, 20-30. You can instantly see if the population leans younger or older.

Syntax Structure:

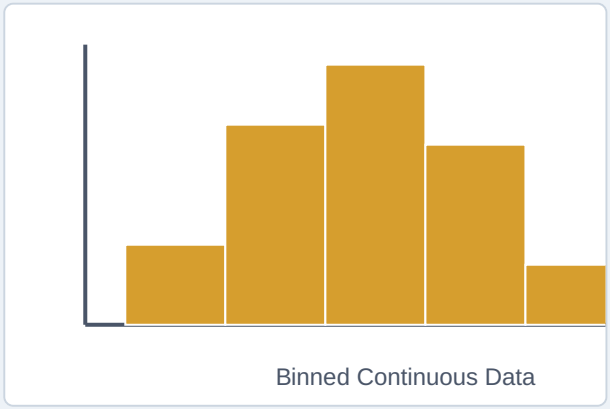
```
hist(numeric_vector, breaks = number_of_bins)
```

R Code Example

```
# Generate 500 random numbers from a normal distribution
set.seed(123)
random_data <- rnorm(500, mean = 50, sd = 10)

# Plot histogram
hist(random_data,
     breaks = 20,
     col = "goldenrod",
     main = "Normal Distribution of Data")
```

Visualization Output




```
(A bell-shaped curve made of touching vertical bars appears, centered
around the number 50.)
```

Line-by-Line Explanation:

`rnorm` creates simulated bell-curve data. `hist()` automatically groups the 500 numbers. `breaks = 20` suggests to R that we want approximately 20 slices across the X-axis for finer detail.

When to Use:

Use histograms to understand the shape, center, spread, and skewness of ONE CONTINUOUS numeric variable.

Common Mistakes:

Confusing Histograms with Bar Charts. Histograms are for continuous numbers (bars touch). Bar charts are for distinct categories (bars have gaps).

Best Practices:

Always experiment with the `breaks` argument. Too few bins hide the true shape of the data; too many bins create a jagged, noisy plot.

Practice Challenge:

Use `hist()` to plot the distribution of the `mpg` column in the `mtcars` dataset.

Comparison: Bar Chart vs Histogram

Feature	Bar Chart (`barplot`)	Histogram (`hist`)
Data Type	Categorical (e.g., Colors, Countries)	Continuous Numerical (e.g., Age, Height)
Visual Appearance	Bars have **gaps** between them.	Bars **touch** each other (continuous).
X-Axis Represents	Distinct, separate groups.	Contiguous 'bins' or ranges of numbers.

5. Box Plots

CONCEPT:
Box plots visualize the statistical summary (median, quartiles, and outliers) of a continuous variable, often grouped by a category.

Deep Dive Explanation:
A box plot (or box-and-whisker plot) packs massive statistical information into a simple shape. The thick line in the middle is the **Median**. The box itself represents the **Interquartile Range (IQR)** (the middle 50% of your data). The lines extending out (whiskers) represent the expected minimum and maximum. Any dots floating beyond the whiskers are mathematically classified as **Outliers**. Base R uses the `boxplot()` function.

Real-Life Example:
Comparing test scores across 3 different classrooms. A box plot instantly shows which class had the highest median, which had the widest spread of scores, and who the extreme over/under-performers were.

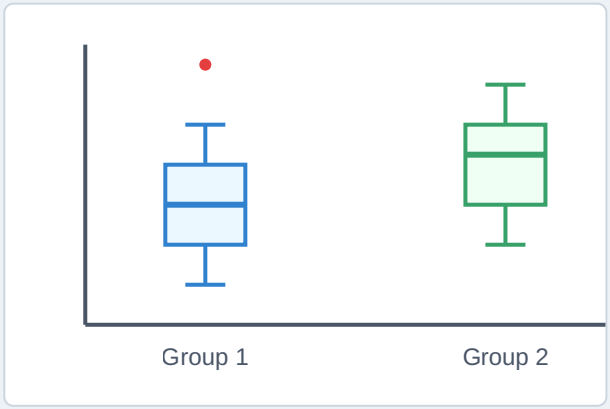
Syntax Structure:

```
boxplot(numeric_vector)                # Single box
boxplot(numeric ~ categorical, data)   # Grouped boxes
```

R Code Example

```
# Using the built-in 'mtcars' dataset
# We plot MPG (continuous) grouped by the number of Cylinders
(categorical)
boxplot(mpg ~ cyl,
        data = mtcars,
        col = c("lightblue", "lightgreen", "lightcoral"),
        main = "MPG by Cylinder Count",
        xlab = "Number of Cylinders",
        ylab = "Miles Per Gallon")
```

Visualization Output



(Three distinct boxes render. The 4-cylinder box is high on the Y-axis, while the 8-cylinder box is low, proving cars with fewer cylinders are more fuel-efficient.)

Line-by-Line Explanation:

The `~` (tilde) formula syntax means 'plot MPG *as explained by* CYL'. R automatically subsets the data into 4, 6, and 8 cylinder groups and draws a separate box for each.

When to Use:

When you need to identify OUTLIERS, or when comparing the distribution of a continuous variable across multiple categories.

Common Mistakes:

Assuming the thick middle line is the Mean. It is strictly the Median (50th percentile).

Best Practices:

Box plots hide the underlying sample size. One box might represent 10 points, another 10,000. It is best practice to note sample sizes if they vary wildly.

Practice Challenge:

Create a boxplot of `hp` (Horsepower) grouped by `gear` (Number of gears) using the `mtcars` dataset.

6. Line Graphs

CONCEPT:
Line graphs connect individual data points with continuous lines, primarily used to visualize trends over time.

Deep Dive Explanation:
In Base R, there is no separate `linegraph()` function. Instead, you use the standard `plot()` function, but you add the argument `type = "l"` (lowercase L for Line) or `type = "b"` (for Both points and lines). R simply draws the dots sequentially and connects them with a rigid line.

Real-Life Example:
Plotting a company's revenue month-by-month over a five-year period to observe growth trends and seasonal dips.

Syntax Structure:

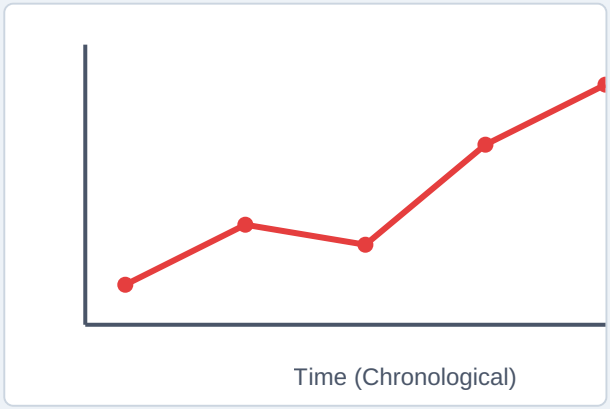
```
plot(x, y, type = "l")
```

R Code Example

```
# Simulating a time series dataset
months <- 1:12
revenue <- c(10, 12, 15, 14, 18, 22, 25, 24, 28, 32, 35, 40)

# Plotting points connected by lines
plot(x = months, y = revenue,
     type = "b",
     pch = 19,
     col = "firebrick",
     main = "Monthly Revenue Growth")
```

Visualization Output



(A graph appears with 12 solid dots connected by a continuous red line moving upward to the right.)

Line-by-Line Explanation:

`type = "b"` plots Both dots and lines. `pch = 19` changes the point character to a solid, filled circle instead of an empty ring. `col` controls the color.

When to Use:

Strictly use line graphs when the X-axis represents an ordered, continuous sequence (like TIME or distance).

Common Mistakes:

Using a line graph for categorical, non-sequential data (e.g., connecting a line from 'Apples' to 'Oranges'). Lines imply a transition from point A to point B.

Best Practices:

Ensure your X-axis data is explicitly sorted chronologically *before* passing it into the `plot()` function. Otherwise, R will draw wild zigzag lines crisscrossing back and forth across the screen.

Practice Challenge:
Create a vector `days <- 1:7` and `visitors <- c(50, 40, 60, 80, 120, 150, 140)`. Plot a line graph.

Comparison: Scatter Plot vs Line Graph

Feature	Scatter Plot	Line Graph
Purpose	To find correlations and outliers.	To track changes and trends over time.
X-Axis Requirement	Any continuous variable (e.g., Weight).	Strictly Ordered/Sequential (e.g., Years).
Base R Type	`plot(x,y, type="p")` (Default)	`plot(x,y, type="l")` or `"b"`

7. Pie Charts

CONCEPT:
Pie charts represent parts of a whole, slicing a circle into portions based on categorical percentages.

Deep Dive Explanation:
Base R provides the `pie()` function. Like `barplot()`, it requires aggregated counts or percentages, not raw data. **Note:** In the data science community, pie charts are heavily

discouraged because human brains are terrible at comparing the angles and areas of slices. A standard bar chart is almost always mathematically superior for comparisons.

> **Real-Life Example:**
> Showing the market share breakdown of 4 competing smartphone brands.

Syntax Structure:

```
counts <- c(25, 60, 15)
pie(counts, labels = c("A", "B", "C"))
```

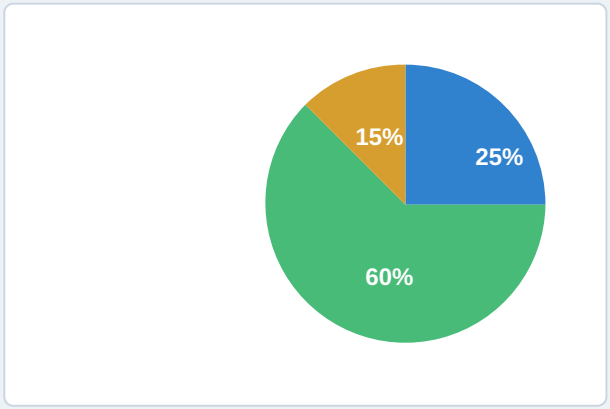
R Code Example

```
# Market share data
shares <- c(45, 30, 15, 10)
brands <- c("Brand A", "Brand B", "Brand C", "Brand D")

# Calculate percentages for labels
pct <- round(shares / sum(shares) * 100)
labels_pct <- paste(brands, "-", pct, "%")

pie(shares, labels = labels_pct,
    col = c("tomato", "skyblue", "gold", "lightgreen"),
    main = "Market Share 2024")
```

Visualization Output



(A pie chart renders with 4 colored slices, explicitly labeled with brand names and perfectly calculated percentages.)

Line-by-Line Explanation:

We manually created percentage labels by dividing each share by the total sum, rounding it, and pasting it together with the brand name. We passed these custom labels into the `pie()` function.

When to Use:

Only use pie charts when comparing a small number of categories (2 to 4) that explicitly add up to 100% of a whole.

Common Mistakes:

Creating a pie chart with 20 tiny, indistinguishable slices. It becomes an unreadable mess of colors.

Best Practices:

If you must use a pie chart, always overlay explicit numeric percentages onto the labels. Never make the viewer guess the size of a slice.

Practice Challenge:

Use `pie()` to plot the distribution `c(70, 30)` with the labels `c("Water", "Land")`.

8. Customizing Base R Plots

CONCEPT:
Plots are useless if the audience cannot read them. Customization involves titles, axis labels, limits, colors, and legends.

Deep Dive Explanation:
Base R plots are drawn like ink on paper. You draw the main plot first, and then you use secondary functions to layer ink on top of it. - `main`, `xlab`, `ylab` control text. - `col` controls color. - `xlim`, `ylim` force the axes to start/stop at specific coordinates. - `legend()` draws a key box explaining the colors.

Real-Life Example:
Changing a plain, black-and-white scatter plot into a branded corporate chart with proper scaling and a descriptive legend.

Syntax Structure:

```
plot(..., main="Title", xlab="X", ylab="Y", col="blue", xlim=c(0, 100))
legend(x="topright", legend=c("A", "B"), fill=c("red", "blue"))
```

R Code Example

```
# Plotting a customized bar chart
sales <- c(100, 150, 120)
regions <- c("North", "South", "East")

barplot(sales, names.arg = regions,
        col = c("darkblue", "darkorange", "darkgreen"),
        main = "Q1 Regional Sales",
        ylab = "Revenue ($K)",
        ylim = c(0, 200)) # Force Y-axis to stretch to 200

# Adding a legend as a separate command
legend("topright", legend = regions,
       fill = c("darkblue", "darkorange", "darkgreen"))
```

Console / Visual Output

```
(A fully polished bar chart with a Y-axis capped at 200 to give the bars breathing room, and a legend box nested perfectly in the top right corner.)
```

Line-by-Line Explanation:

The `names.arg` applies text to the bottom of the bars. `ylim = c(0, 200)` manually stretched the vertical axis. The `legend()` function was called *after* the plot to stamp the key onto the canvas.

When to Use:

Apply customizations to EVERY final plot intended for an external audience or report.

Common Mistakes:

Forgetting that Base R draws like a pen. If you add a legend and it overlaps with your data bars, you cannot 'undo' it. You must rerun the entire `barplot()` code from scratch.

Best Practices:
Ensure text labels are highly descriptive. Never submit a chart with default variable names like `mtcars$mpg` as the axis label.

Practice Challenge:
Recreate a basic `plot(1:10)` but make the points green, add a title, and force the Y-axis to scale from 0 to 20 using `ylim = c(0,20)`.

Anatomy of a Customized Plot

Parameter	Action	Example Code
main	Main Title (Top Center)	`main = "Annual Sales Report"`
xlab / ylab	X and Y Axis Labels	`xlab = "Months", ylab = "Revenue"`
col	Color of bars/lines/points	`col = "steelblue"`
xlim / ylim	Forces axis limits (Bounds)	`ylim = c(0, 100)`
legend()	Adds a key box identifying colors	`legend("topleft", legend=c("A"))`

9. Introduction to ggplot2 & Grammar of Graphics

> **CONCEPT:**
> ggplot2 is the absolute industry standard for R visualization, built on a philosophy of layering components.

Deep Dive Explanation:

Base R plotting is functional, but it relies on scattered parameters. `ggplot2` (part of the Tidyverse) revolutionizes this by using Leland Wilkinson's **'Grammar of Graphics'**. It states that any graphic is composed of independent layers. You start with raw Data, map it to a canvas using Aesthetics (X, Y), add a geometric shape (Geom), and finally tweak the Coordinates and Themes. You literally add these layers together using the `+` symbol.

Syntax Structure:

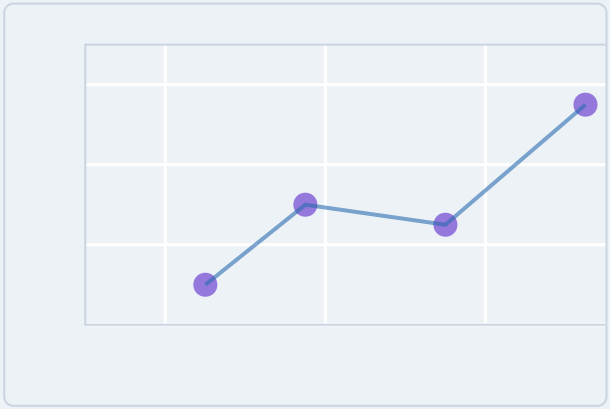
```
library(ggplot2)
ggplot(data, aes(x, y)) + geom_shape() + theme_layer()
```

R Code Example

```
library(ggplot2)

# The fundamental ggplot structure
ggplot(data = mtcars, mapping = aes(x = wt, y = mpg)) +
  geom_point()
```

Visualization Output



(A highly polished, modern scatter plot appears with an automatic light-grey background, white gridlines, and clearly mapped points.)

Line-by-Line Explanation:

The `ggplot()` function creates the blank canvas and sets the rules (`aes`). The `+` symbol links the next layer. `geom_point()` tells R to draw dots according to the rules established on the canvas layer.

When to Use:

Use `ggplot2` for all complex, multi-layered, or presentation-quality data visualizations.

Common Mistakes:

Using the pipe `%>%` instead of the plus `+` to add layers. `dplyr` uses `%>%` to chain logic; `ggplot2` explicitly uses `+` to add visual layers.

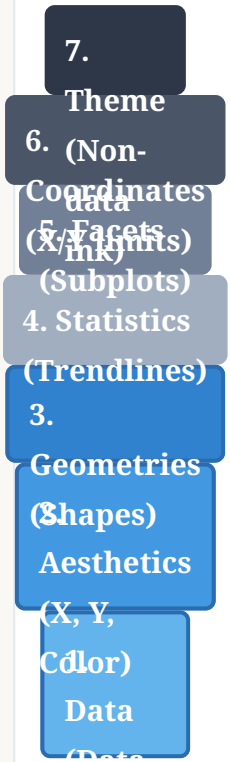
Best Practices:

Always structure your code with the `+` at the end of the line, indenting the subsequent layer on the next line for perfect readability.

Practice Challenge:

Load `ggplot2`. Run just `ggplot(data = mtcars)`. What happens? (Hint: It should just be a blank grey canvas).

The Grammar of Graphics Layers



You build a ggplot from the bottom up. Layers 1-3 are mandatory for any plot.

Comparison: Base R vs ggplot2

Feature	Base R `plot()`	`ggplot2`
Philosophy	Pen on paper (cannot erase/undo).	Layered objects (can add/remove layers safely).
Syntax Style	Comma-separated arguments.	Additive layers linked by +.
Legends/ Colors	Manual, tedious calculation required.	100% Automatic mapping via `aes()`.

10. ggplot2 Aesthetics (aes)

> **CONCEPT:**
> Aesthetics act as the mapping bridge, linking raw variables in your dataset to visual properties on the chart.

Deep Dive Explanation:

The `aes()` function is the brain of ggplot. It answers the question: 'How do the columns in my data frame translate to ink on the screen?' You map variables not just to the X and Y axes, but also to `color`, `size`, `shape`, and `alpha` (transparency). If you map a categorical variable to `color` inside `aes()`, ggplot automatically colors the points, groups them, and instantly generates a perfect legend.

Syntax Structure:

```
ggplot(data, aes(x = col1, y = col2, color = col3))
```

R Code Example

```
library(ggplot2)

# Map continuous variables to X and Y.
# Map a categorical variable (factor(cyl)) to color and shape.
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl),
shape = factor(cyl))) +
  geom_point(size = 3) +
  labs(title = "Car Performance", color = "Cylinders", shape =
"Cylinders")
```

Console / Visual Output

```
(A scatter plot renders where 4-cylinder cars are red circles, 6-cyl are
green triangles, and 8-cyl are blue squares. A flawless legend appears
automatically on the right.)
```

Line-by-Line Explanation:

By wrapping `cyl` inside `factor()`, we forced ggplot to treat the numbers 4, 6, 8 as distinct groups rather than a continuous gradient. We put `size = 3` *outside* of `aes()` because we wanted ALL dots to be size 3, not mapped to a variable. The `labs()` function cleans up the titles.

When to Use:

Use inside `aes()` when mapping a visual property to dynamic DATA. Use outside `aes()` when setting a fixed visual property for the whole chart.

Common Mistakes:

Putting `color = "blue"` inside `aes()`. It will create a useless legend that says 'blue'. Fixed colors must go inside the geom, e.g., `geom_point(color = "blue")`.

Best Practices:
Leverage multiple aesthetics (e.g., color AND shape) simultaneously to make your charts accessible to colorblind viewers.

Practice Challenge:
Create a scatter plot of `disp` vs `hp`. Inside `aes()`, map the continuous variable `mpg` to `color`. Watch how R creates a gradient scale instead of distinct categories.

11. ggplot2 Geometries (geom_*)

CONCEPT:
Geometries dictate the actual physical shapes drawn on the canvas (points, bars, lines, boxes).

Deep Dive Explanation:
Once your canvas and aesthetics are set, you choose a Geometry layer to interpret them. - `geom_point()` creates a scatter plot. - `geom_col()` creates a bar chart (if providing exactly X and Y). - `geom_histogram()` creates a histogram (requires only X). - `geom_boxplot()` creates a box plot. - `geom_line()` creates a line graph. You can literally stack multiple geoms on top of each other! You can have a line graph with points mapped exactly on top of it.

Syntax Structure:

```
... + geom_point() + geom_smooth(method = "lm")
```

R Code Example

```
library(ggplot2)

# Stacking two geometries: Points and a Line of Best Fit
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(color = "darkblue", size = 2) +
  geom_smooth(method = "lm", color = "red", se = FALSE) +
  labs(title = "MPG vs Weight with Trendline")
```

Console / Visual Output

```
(A standard scatterplot is drawn, but sliced perfectly through the middle
is a red linear regression trendline.)
```

Line-by-Line Explanation:

The `geom_smooth()` layer automatically calculated the mathematics for a linear model (`"lm"`) based on the existing X and Y aesthetics and drew the red line over the points. `se = FALSE` hides the grey confidence interval ribbon.

Common Mistakes:

Using `geom_bar()` when you have pre-counted data. `geom_bar()` wants to count the raw data itself. If you already counted it, you MUST use `geom_col()`.

Best Practices:

Experiment with stacking. A `geom_boxplot()` overlayed with a slightly transparent `geom_jitter()` allows viewers to see both the statistical summary and the exact distribution of every underlying data point.

Practice Challenge:

Swap `geom_point()` for `geom_line()` in a basic ggplot. See how the visual representation changes entirely.

Common ggplot2 Geometries

Geometry	Equivalent Base R	Required Data Types
`geom_point()`	`plot()` (Scatter)	Continuous X, Continuous Y
`geom_bar()`	`barplot(table(x))`	Categorical X (Auto-counts Y)
`geom_col()`	`barplot(y)`	Categorical X, Continuous Y (Pre-counted)
`geom_histogram()`	`hist()`	Continuous X (Auto-bins Y)
`geom_boxplot()`	`boxplot()`	Categorical X, Continuous Y
`geom_line()`	`plot(type="l")`	Continuous/Date X, Continuous Y

12. Themes and Facets (ggplot2)

> **CONCEPT:**
> Facets split data into multiple sub-charts. Themes completely alter the non-data visual elements (backgrounds, fonts, gridlines).

Deep Dive Explanation:

If you have data spanning 4 years, plotting it all on one chart can look messy. **Faceting** (`facet_wrap(~ variable)`) instantly splits your single chart into 4 perfectly aligned mini-charts side-by-side. **Themes** polish the final product. Base `ggplot2` has a grey

background, but adding `+ theme_minimal()` makes it clean and modern. You can customize literally every pixel using the `theme()` function.

Syntax Structure:

```
... + facet_wrap(~ column_name) + theme_minimal()
```

R Code Example

```
library(ggplot2)

# Create a faceted, themed plot
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(color = "purple") +
  facet_wrap(~ cyl) +                # Split into 3 mini-charts by
cylinder
  theme_bw() +                       # Apply a clean black-and-
white theme
  labs(title = "Faceted Engine Analysis")
```

Console / Visual Output

```
(Instead of one chart, three miniature scatter plots appear side-by-side
(labeled 4, 6, 8) sharing the same axis scales, set against a crisp white
background with subtle gridlines.)
```

Line-by-Line Explanation:

The `~ cyl` syntax tells `facet_wrap` to split the dataset by the unique values in the `cyl` column. `theme_bw()` overrides the default grey background.

Common Mistakes:

Forgetting the tilde `~` inside `facet_wrap()`. R will throw an error. The tilde means 'faceted *by*'.

Best Practices:

Use `theme_minimal()`, `theme_light()`, or `theme_classic()` for modern, professional reports. The default grey theme is iconic but often considered too heavy for printed publications.

Practice Challenge:

Add `+ theme_dark()` to any of your previous ggplot charts and observe the aesthetic shift.

IT STUDY HUB

HANDS-ON PROJECT

IT STUDY HUB

COMPLETE DATA VISUALIZATION REPORT

Problem Statement:
You are a data analyst for a regional electronics retailer. Management has handed you a curated dataset of their recent store performance. They want a complete visual report answering three core questions: What is the distribution of our sales revenue? How do our stores compare in total performance? Is there a trend in sales over the last 12 months? You will use modern `ggplot2` pipelines to construct this report.

1. Setting up the Data Overview

```
library(ggplot2)
library(dplyr)

# Simulated Business Data
sales_data <- tibble(
  Month = factor(month.abb, levels = month.abb), # Jan to Dec ordered
  Revenue = c(20, 25, 28, 22, 35, 45, 42, 50, 55, 60, 80, 95), # In $1000s
  Store_Type = c(rep("Urban", 6), rep("Suburban", 6)),
  Foot_Traffic = c(500, 600, 750, 550, 800, 1100, 1050, 1200, 1300, 1400, 1800, 2100)
)
```

2. Scatter Plot: Is Foot Traffic driving Revenue?

```
ggplot(sales_data, aes(x = Foot_Traffic, y = Revenue, color =
Store_Type)) +
  geom_point(size = 4, alpha = 0.8) +
  geom_smooth(method = "lm", se = FALSE, color = "black",
linetype = "dashed") +
  theme_minimal() +
  labs(title = "Revenue vs Foot Traffic", x = "Monthly
Visitors", y = "Revenue ($K)")
```

Interpretation: There is a massive, positive linear correlation. As foot traffic increases, revenue scales predictably. Furthermore, Suburban stores exclusively cluster at the high end of both metrics, identifying them as the top performers.

3. Bar Chart: Total Revenue by Store Type

```
# Summarize data first
store_summary <- sales_data %>%
  group_by(Store_Type) %>%
  summarize(Total_Rev = sum(Revenue))

# Plot pre-counted data using geom_col
ggplot(store_summary, aes(x = Store_Type, y = Total_Rev, fill =
Store_Type)) +
  geom_col(width = 0.6) +
  theme_classic() +
  labs(title = "Total Revenue by Region", x = "", y = "Total
Revenue ($K)")
```

Interpretation: The Suburban stores generated roughly double the total cumulative revenue of the Urban stores across the measured period.

4. Line Graph: Sales Trend over Time

```
# Group = 1 tells ggplot to draw a single line across
categorical X values
ggplot(sales_data, aes(x = Month, y = Revenue, group = 1)) +
  geom_line(color = "steelblue", size = 1.5) +
  geom_point(size = 3, color = "darkblue") +
  theme_light() +
  labs(title = "2024 Monthly Revenue Trend", x = "Month", y =
"Revenue ($K)")
```

Interpretation: Revenue experienced stagnant, flat growth from January to April. A massive breakout trend began in May, compounding exponentially through the holiday season (Nov/Dec).

Final Data-Driven Insight Report:

Based on the generated visualizations, the business is scaling healthily with a heavy Q4 bias. However, Urban stores are vastly underperforming Suburban locations. The scatter plot confirms this is a **Volume problem, not a Conversion problem**: Urban stores simply lack Foot Traffic. Marketing spend should be aggressively diverted to increase raw foot traffic in Urban locations.

ASSESSMENTS & INTERVIEW PREP

15+ Practice Questions

Test your fundamental knowledge of data visualization in R.

1. What Base R function draws a basic scatter plot?
2. If you want to view the distribution of a single numeric column (like Age), which plot should you use?
3. What function must you use BEFORE `barplot()` if you are given a raw list of categorical text strings?
4. In a Box Plot, what does the thick line in the absolute center of the box represent?
5. How do you tell Base R's `plot()` function to draw a line instead of dots?
6. Why are pie charts generally discouraged in the data science community?
7. What argument inside a base `plot()` changes the main title at the top?
8. How do you manually force the Y-axis of a base plot to range from 0 to 100?
9. What package must you load to use `ggplot()`?
10. What are the three mandatory layers required to render any visual `ggplot2` chart?
11. If you want to map point colors dynamically based on a variable in your dataset, does `color = variable` go inside or outside `aes()`?
12. Which `geom_` creates a bar chart when you provide both an X and Y axis?
13. What does `geom_smooth(method = "lm")` add to a scatter plot?
14. What function allows you to split a single ggplot into multiple side-by-side mini charts?
15. Which `ggplot2` function removes the default grey background and applies a clean, modern white background?

IT STUDY HUB

20+ Viva Questions (With Answers)

These questions are designed for rapid-fire university viva or oral examinations.

Q1: What is Anscombe's Quartet?

Answer: A dataset proving that completely different geometric data can have identical summary statistics (mean, variance), highlighting the absolute necessity of visualizing data.

Q2: Which axis is the dependent variable typically plotted on?

Answer: The Y-axis (Vertical).

Q3: What data types are required for a Scatter plot?

Answer: Two continuous (numeric) variables.

Q4: What data types are required for a Bar chart?

Answer: One categorical variable (X) and one aggregated numeric variable (Y).

Q5: What is the visual difference between a Bar chart and a Histogram?

Answer: Bar charts have gaps between bars (distinct categories). Histograms have touching bars (continuous numeric bins).

Q6: What do the 'whiskers' on a box plot represent?

Answer: The expected minimum and maximum spread of the data, excluding mathematical outliers.

Q7: What argument in `plot()` changes the axis labels?

Answer: `xlab` and `ylab`.

Q8: How do you add a legend to a Base R plot?

Answer: By calling the `legend()` function as a completely separate line of code after the plot is drawn.

Q9: What does 'Grammar of Graphics' mean?

Answer: A philosophy stating that graphics are built systematically via independent, stackable layers (Data + Aesthetics + Geoms).

Q10: What symbol connects layers in `ggplot2`?

Answer: The plus sign `+`.

IT STUDY HUB

Q11: What does `aes()` stand for and what does it do?

Answer: Aesthetics. It maps columns in your data frame to visual properties on the canvas (X, Y, Color, Size).

Q12: If I type `geom_point(color="red")` outside of `aes()`, what happens?

Answer: Every single dot on the scatter plot will turn red. It is a fixed assignment, not a dynamic data mapping.

Q13: What happens if I type `aes(color="red")`?

Answer: R treats "red" as a category name, not a literal color. It will draw a legend labeled "red" but assign it a default ggplot color (like salmon pink).

Q14: What is the difference between `geom_bar()` and `geom_col()`?

Answer: `geom_bar()` wants to count the raw data for you (needs only X). `geom_col()` expects you to provide pre-counted heights (needs X and Y).

Q15: How do you overlay a line on top of points in ggplot?

Answer: By adding both geoms sequentially: `... + geom_point() + geom_line()`.

Q16: What does `facet_wrap(~ category)` do?

Answer: It fragments one large plot into multiple smaller subplots based on the unique values of that category.

Q17: What does the `alpha` aesthetic control?

Answer: Transparency of the geometric object (from 0 invisible to 1 completely solid).

Q18: Why is `alpha` useful in scatter plots?

Answer: It solves 'overplotting' by making dense clusters of overlapping dots appear darker.

Q19: What does `theme_minimal()` do?

Answer: It strips away the heavy grey background of default ggplot, leaving a clean white canvas with light gridlines.

Q20: Why are Line graphs strictly for Time Series or ordered sequences?

Answer: Because connecting dots implies a transition from point A to point B. Connecting 'Apples' to 'Oranges' is logically meaningless.

IT STUDY HUB

15+ Corporate Interview Questions

Prepare for data analyst technical rounds with these comprehensive visualization logic questions.

1. A stakeholder brings you a raw dataset of 100,000 global housing transactions. Walk me through the first three visualizations you create to explore this data.

2. Explain the fundamental architectural difference between Base R plotting and `ggplot2` plotting.

3. You create a bar chart, but the bars are completely out of order. How do you force R to display them logically (e.g., Small, Medium, Large)?

4. I have a dataset of customer ages. Why would I explicitly choose to use a Histogram over a Box Plot? When would I reverse that choice?

5. Your ggplot scatter plot has 500,000 dots, resulting in a giant black blob. What three techniques can you use to make the density readable?

6. Explain exactly what the `aes()` function is doing under the hood. Why does `color = variable` inside `aes` automatically generate a legend, but outside `aes` it does not?

7. What is the mathematical difference between `geom_bar()` and `geom_col()`? Provide a real-world scenario where you must use `geom_col()`.

8. A line graph tracking monthly revenue over three years looks like a giant, messy zigzag scribbling backward and forward. What went wrong with the data?

9. Explain how you would use `facet_wrap()` to answer a business question comparing Q1, Q2, Q3, and Q4 sales.

10. Why is the Pie Chart widely considered an anti-pattern in the data science community, and what visualization should replace it?

11. In a Box Plot, a stakeholder asks you what the 'dots' above the top whisker represent. Explain this to them mathematically.

12. You write a ggplot code block and end a line with the pipe `%>%` instead of a `+`. What happens, and why?

IT STUDY HUB

13. How do you override the default legend title generated by ggplot's `aes(color = Region)`?

14. What does `geom_smooth(method = 'lm')` accomplish? Why is it vital for executive reporting?

15. A chart looks perfect in RStudio, but when exported to a PDF report, the text is microscopic. How do `Themes` help fix this globally?

IT STUDY HUB

MODULE WRAP-UP

Module Summary

In Module 6, you successfully transitioned from Data Wrangling to Data Storytelling. You learned that Anscombe's Quartet proves summary statistics are insufficient; data must be seen. We began with Base R, learning to rapidly deploy Scatter, Bar, Histogram, Box, and Line plots to instantly verify the shape and correlations of raw data. You learned the critical distinction between categorical mapping (Bars/Boxes) and continuous mapping (Scatters/Histograms).

We then elevated to presentation-grade analytics using `ggplot2`. By mastering the Grammar of Graphics, you learned to construct charts modularly. You mapped dynamic data using Aesthetics (`aes`), rendered shapes using Geometries (`geom_*`), stacked statistical trendlines (`geom_smooth`), and polished the final output using professional Themes and Facets. Your pipelines can now take raw data and output beautiful, compelling corporate intelligence.

IT STUDY HUB

Key Terms Dictionary

Term	Definition
Scatter Plot	A plot of continuous X vs continuous Y to identify correlation.
Histogram	A distribution plot grouping continuous numbers into touching 'bins'.
Box Plot	A statistical summary plot showing median, IQR, and outliers.
Grammar of Graphics	A framework for building plots sequentially using independent layers.
Aesthetics (aes)	The mapping of a dataset variable to a visual property (e.g., color, X-axis).
Geom	The geometric object drawn to represent the data (e.g., point, line, bar).
Facet	Splitting one massive plot into multiple aligned subplots based on a category.
Overplotting	When massive amounts of data points overlap, hiding the true distribution.

Quick Revision Checklist

- ✅ Can you match variable types (Categorical vs Continuous) to their proper chart?
- ✅ Do you know the difference between `hist()` and `barplot()`?
- ✅ Do you understand the difference between Data mapping (inside `aes`) and Fixed settings (outside)?
- ✅ Do you know to use a + instead of a pipe %>% to build ggplot layers?

- ✅ Can you explain what the thick middle line in a Box Plot represents?

Visualization Coding Challenges

Challenge 1: The Base Explorer
Using the built-in `iris` dataset, write a Base R `plot()` command comparing `Sepal.Length` to `Petal.Length`. Make the points red, and add a custom main title.

Challenge 2: The Grammar Architect
Rebuild the plot from Challenge 1 using `ggplot2`. Map the categorical `Species` column to the `color` aesthetic. Add a `theme_minimal()` layer.

Challenge 3: The Storyteller
Using `ggplot2`, create a `geom_boxplot()` mapping `Species` on the X-axis and `Sepal.Length` on the Y-axis. What does the visualization instantly tell you about the Virginica species compared to Setosa?

IT STUD

WHAT COMES NEXT:

MODULE 7 — ADVANCED DATA MANIPULATION WITH TIDYVERSE

You can now ingest, clean, and beautifully visualize data. Next, we will return to the Tidyverse to unlock advanced manipulation: joining multiple datasets together, pivoting complex tables, and performing high-level grouped aggregations.

IT STUD